

Agentic AI
Foundation

AI야 GitOps를 만나볼래?

(그래서 오래오래 행복하게 잘 살았을까요?)

Hoon Jo | CNCF & AAIF Ambassador

GitAIOps: Git + AI + Ops

GitOps부터 말씀드리겠습니다. 원하는 상태를 Git에 선언하면 Argo CD가 클러스터를 그 상태로 맞춥니다.

누가 클러스터를 직접 고치면 다시 선언된 상태로 되돌려 버립니다.

그리고 중요한 것은 이를 위한 저장소가 SSOT(Single Source Of Truth, 단일 진실 공급원)입니다.

여기에 **AI**를 더하면, 사람이 하던 **계획·검토·기록·되돌리기의 절차 안에서 실제 작업을 AI가 맡습니다.**

CLAUDE.md

항상 지켜야 할 규칙

매 세션 로드

claude-context/

현재 아키텍처의 전체 그림

필요할 때 참조

docs/ (ADR)

결정의 누적 기록

사람과 AI가 함께 검토

위의 내용이 오늘 내용의 결과물이고, 이 형태를 가지기까지 그리고 그 이후에 이야기를 해보겠습니다.

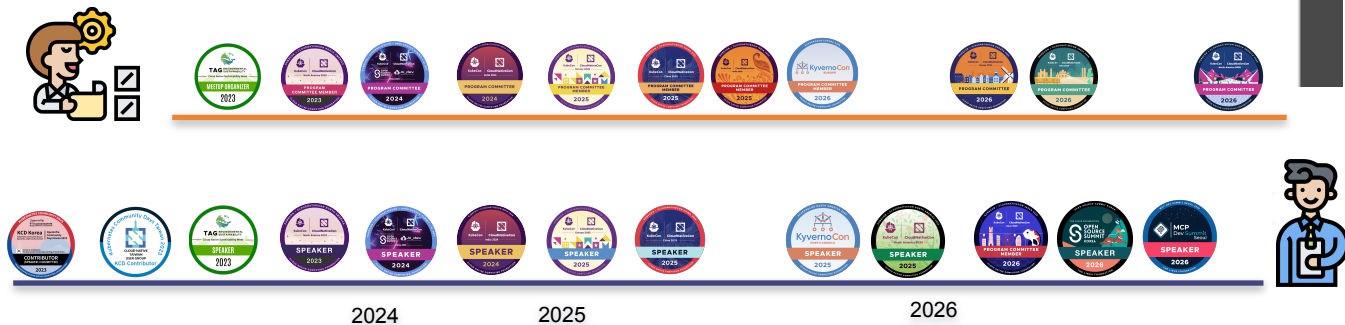


Agentic AI
Foundation



AMBASSADOR

Agentic AI Foundation



1부. GitAIOps로 행복한 이야기 다섯



Agentic AI
Foundation

대규모 이관을 AI와 함께 해냈습니다

노드 25대 · 파드 429개 · Helm 릴리스 15개 규모의 클러스터를 새 버전으로 이관했습니다.

계획은 사람이

무엇을 어떤 순서로 옮길지 문서로 작성합니다

실행은 AI가

문서를 읽고 명령을 만들어 실행합니다. 순서는 문서로 고정되어 건너뛸 수 없습니다

안전장치

실행 전에 롤백 명령을 먼저 준비하고 결과는 다시 문서로 기록합니다

혼자 감당할 수 없던 규모의 일이, 혼자 감당되는 규모가 되었습니다.

무엇을 해도 Git에 남습니다

AI에게 클러스터 쓰기 권한은 주지 않았습니다. git add / commit / push만 열었습니다.
Git에 쓰면 Argo CD가 배포하고 잘못되면 revert 하면 되었습니다. (실수가 있어도 쉽게 복구!)

	kubectl에 권한을 줬다면	git에만 권한을 주니
기록	셀 히스토리	커밋
검토	없음	diff
되돌리기	기억나는 대로	revert

AI가 실수해도 배포 전에 diff에서 걸리고 배포 뒤에도 revert 하나로 되돌릴 수 있습니다.

AI가 무엇을 했는지 커밋 목록으로 언제든 확인할 수 있고, 위에 적혀 있는 것처럼 되돌리는 것도 간단했습니다.

진행한 것을 기록하고, 다음에 또 사용했습니다

컨텍스트가 압축되면 AI는 기억을 잃어버리니 다음과 같이 기록을 해두었습니다.

행동 규칙

CLAUDE.md

아키텍처 문서

현재 구조의 전체 그림

교훈 46개

번호 붙은 실수 기록

교훈의 예: "Argo CD가 관리하는 앱은 클러스터가 아니라 Git을 고친다" (46번)

두 번째 프로젝트는 이 구조를 그대로 복사해서, 처음부터 문서를 만들지 않고 바로 작업을 시작할 수 있었습니다.



남은 시간으로 만든 것 1: 이슈 알림 그리고 자동 분석

알림이 오면 AI가 로그·메트릭 추이·트레이스·이벤트·배포 상태·큐 지표·과거 90일까지
8가지 신호를 모아 분석하고 원인과 해결 방법을 Slack으로 보냅니다. 나중에는 문자 발송까지 구현했습니다.

월 1~2달러

분석 비용

첫날

실제 장애 분석 성공

93%

알림 46종 중 깊은 분석

프롬프트에는 이런 줄도 있습니다:

Don't say "로그 확인 필요" if the cause is already clear

남은 시간으로 만든 것 2: 개발 환경 자동 정지·기동

개발 환경은 밤과 주말에는 쓰는 사람이 없습니다. 그런데 켜 두면 비용은 그대로 나갑니다.

평일 20시

자동 정지

아침 8시

자동 기동 (약 12분)

주말

내내 꺼짐

비용은 월 수백 달러가 줄었습니다.

아침에 출근해서 그대로 쓰면 됩니다. 개발자는 차이점을 알 수가 없었습니다.

그래서 저희는 행복하게
오래오래 잘 살았습니다.



Agentic AI
Foundation

2부. 뒷이야기 다섯

끝난 게 끝난 게 아니었습니다

이관이 끝났는데, 일이 끝나지 않았습니다

무슨 일

배포 완료 후 운영에서 문제가 이어졌습니다.
메모리 부족 2건, 로그 수집 누락. 하루에 5건을 기록한 날도 있습니다.

왜

이관 점검은 구성이 맞는지까지만 봅니다. 메모리·용량 문제는 트래픽이 쌓여야 나타납니다

어떻게

고치고, 기록하고, 반복될 일은 도구로. 그 주에 운영용 스킬 5개를 만들었습니다

이관 완료 커밋을 찍은 날 일이 끝난 줄 알았는데, 종류가 다른 일이 시작된 거였습니다.

만드는 일과 운영하는 일은 다른 영역입니다. (즉 계속 관심을 두고 관찰하고 돌봐줘야 합니다.)

남는 건 좋은데, 남은 것도 관리가 필요했습니다

커밋 250개 중 166개(66%)가 문서 커밋. 상태 문서 하나를 115번 고쳤습니다.

큰 파일이 쌓여 복제가 제한 시간을 넘겼고, 지우려면 이력 전체를 다시 쓰는 작업까지 필요했습니다.

갱신은 절차로

스킬이 총 순서대로 증류하고 커밋까지

커지면 쪼개기

658줄을 역할별 5개 파일로

큰 파일은 밖에

스냅샷·로그 덤프 원본은 저장소 밖, 안에는 분석 결과와 발체만

성격으로 나누기

매니페스트 저장소(2,255커밋)에는 문서를 두지 않음

다 남는 건 정말 좋은 점입니다. 대신 남은 것을 관리하는 일까지가 **GitAIOps**입니다.



기록되는 건 좋은데, 기록된 것은 실제와 어긋납니다

아무도 문서를 고치지 않았는데 문서 내용이 실제와 어긋납니다.

점검마다 pod 141 → 142, apps 65 → 64, rules 32 → 43. 여섯 번 점검에 매번 새 어긋남이 나왔습니다.

문서는 가만히 있는데 클러스터는 운영으로 계속 변합니다. AI가 읽는 문서가 곧 AI의 현실 인식입니다.

점검 시점을 정한다

큰 변경 전 · 사고 후 · "전체 재점검" 한마디로 문서와 실물 대조

두 곳에 두지 않는다

값은 한 곳에 두고 나머지는 참조로

확인한 날짜를 적는다

날짜 지난 상태 문서는 믿지 않고, 확인 명령을 한 줄 함께

규칙 파일로 강제한다

CLAUDE.md에 "변경하면 저장소 반영과 commit·push까지" 강제



알림의 절반은 AI가 분석하지 못하고 있었습니다

분류	규칙	심층	커버리지
파드 상태	8	8	100%
리소스	10	6	60%
큐(Kafka)	6	0	0%
배포(Argo CD)	5	0	0%
노드	5	3	60%
네트워크	4	2	50%
기타	8	5	63%
합계	46	24	52%

자동으로 오는 알림 46종 중

AI가 분석하는 건 24종

필요한 알림인데 신호를 수집하지 않아 분석이 안 되
던 것들(큐·배포)이 있었습니다.

수집을 추가하고 추이·과거 90일을 붙여서

52% → 93%

코드 809 → 1,853줄

1부의 93%는 이렇게 채워서 만든 숫자입니다. 필요한데 분석되지 않던 알림을 찾아 채웠습니다.

꺼지는 건 쉬웠는데, 켜질 때가 문제였습니다

정지는 문제가 없었습니다. 문제는 아침 시작할 때였습니다.

켜지는 중에 일시적인 이상 상태로 알림이 발생하고, 이를 시스템이 장애로 판단해서, 매일 아침 알림이 연달아 왔습니다.

사람이 판단해서 껏습니다

기동 시점에만 울리는 5종류 · 평일 07:55~08:30만

Pod Not Ready · Target Down · App Health 등

그대로 두었습니다

그 시간에도 올려야 하는 것

CPU/Mem 95% · 5xx · OOMKilled

어떤 알림이 기동 소음이고 어떤 알림이 진짜 장애인지는 사람이 정해야 했습니다.

자동화가 만드는 정상 상태를 기계가 스스로 구분하지는 못했습니다. 11번 돌려 보고 나서야 안정됐다고 기록했습니다.

행복 다섯, 뒷이야기 다섯

행복

대규모 이관을 AI와 해냈다

무엇을 해도 Git에 남는다

배운 것이 기록된다

알림 93% 자동 분석

자동으로 꺼지고 켜진다

뒷이야기

끝난 줄 알았는데 운영이 시작됐다

남은 것도 관리가 필요했다

기록된 것은 실제와 어긋난다

절반은 분석하지 못하고 있었다

켜질 때는 사람이 개입해야 했다

왼쪽만 보면 우리가 꿈꾸는 이상향, 오른쪽만 보면 실제 우리가 만나는 일입니다.



Agentic AI
Foundation

이것이 행복이다!! 행복편

1 AI와 대규모 이관

AI와 함께하는 원활한 대규모 이관



2 Git에 남는 모든 것

우리의 모든 코드가 Git 타임라인에 완벽하게 기록됨



3 기록되는 배움

배운 지식이 자동으로 기록되고 저장됨



4 93% 자동 분석

수많은 알림이 93% 정확도로 자동 분석됨



5 자동 켜기/끄기

필요에 따라 장치, 자동으로 꺼지고 켜짐



이것이 파멸이다!! 릿이야기편

1 운영의 시작

끝난 줄 알았는데, 더 힘든 운영이 시작됨



2 관리 필요한 잔재

완료된 프로젝트의 잔재가 방치되어 관리가 필요함



3 실제와 다른 기록

기록된 데이터와 실제 결과가 서로 맞지 않음



4 분석 실패 절반

알림의 절반은 제대로 분석되지 않음



5 수동 개입 필수

시스템이 켜질 때마다 사람이 수동으로 개입해야 함



왼쪽만 보면 우리가 꿈꾸는 이상향, 오른쪽만 보면 실제 우리가 만나는 일 입니다.

그럼에도 불구하고

몇 사람이 나눠서 해야 하던 관리를 이제 1명이 할 수 있게 되었고,
오히려 시간 여유가 생겨서 개선까지 할 수 있게 되었습니다.

여러 명 → 1명

운영 인원

월 1~2달러

분석 비용

월 수백 달러 절감

환경 비용

배포 뒤에 생기는 이런 일들을 모두 처리하고도, 전체 비용과 효율은 이전과 비교가 안 됩니다.

관리는 사라진게 아니라 여전히 필요한 일이지만 1사람이 관리할 수 있는 범위가 달라졌습니다.

운영은 그 다음 날도 그리고 그 다음 달도
계속됩니다. 😊

대신 이제는 남은 시간에 필요한 도구를 만들 여유와 업무의 범위를 확장할 수 있습니다.

감사합니다

질문은 세 번째 발표가 끝나고 이어지는 패널 토크에서 받겠습니다.

잠시 쉬었다가 세 번째 발표로 이어집니다.



kuberneteslab.dev

블로그·발표 자료



github.com/sysnet4admin

실습·강의 저장소



github.com/gitaiops

GitAIops 조직 (데모 실물)



**Agentic AI
Foundation**